Concept of Partitioning

Partitioning enables tables and indexes to be subdivided into individual smaller pieces. Each piece of the database object is called a partition. A partition has its own name, and may optionally have its own storage characteristics.

Partitioning allows tables and indexes to be divided into small parts for easy management inside database. You can create small portions of very large tables inside database which makes data access more simple and manageable.

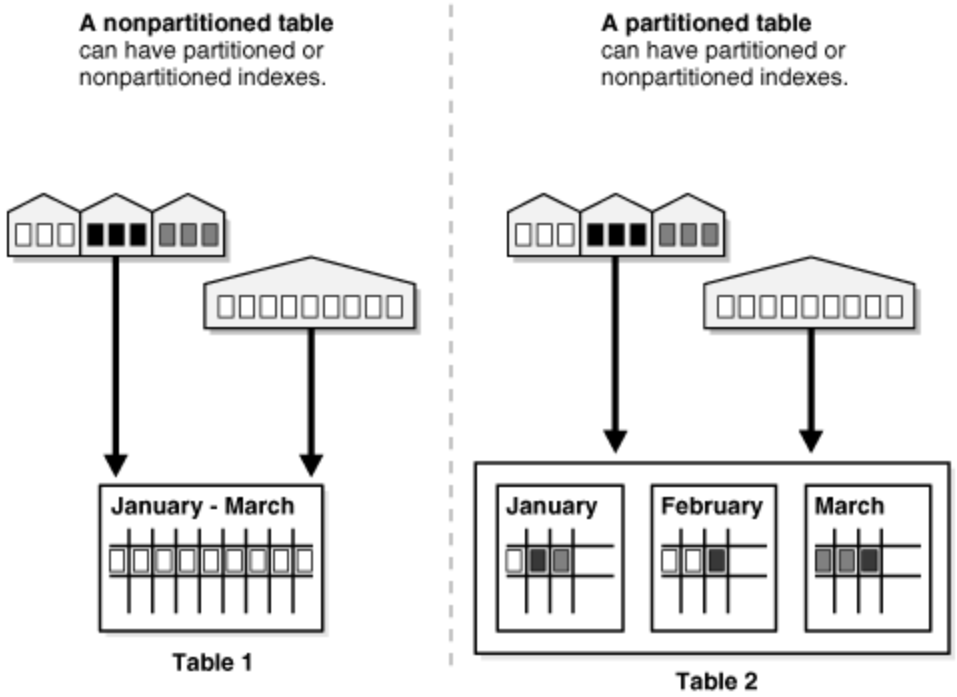
Partitioning offers these advantages:

- Partitioning enables data management operations such data loads, index creation and rebuilding, and backup/recovery at the partition level, rather than on the entire table. This results in significantly reduced times for these operations.
- Partitioning improves query performance. In many cases, the results of a query can be achieved by accessing a subset of partitions, rather than the entire table.
- Partitioning increases the availability of mission-critical databases if critical tables and indexes are divided into partitions to reduce the maintenance windows, recovery times, and impact of failures.
- Partitioning can be implemented without requiring any modifications to your applications. For example, you could convert a nonpartitioned table to a partitioned table without needing to modify any of the `SELECT` statements or DML statements which access that table. You do not need to rewrite your application code to take advantage of partitioning.

When to Partition a Table

Here are some suggestions for when to partition a table:

- Tables greater than 2GB should always be considered for partitioning.
- Tables containing historical data, in which new data is added into the newest partition. A typical example is a historical table where only the current month's data is updatable and the other 11 months are read only.



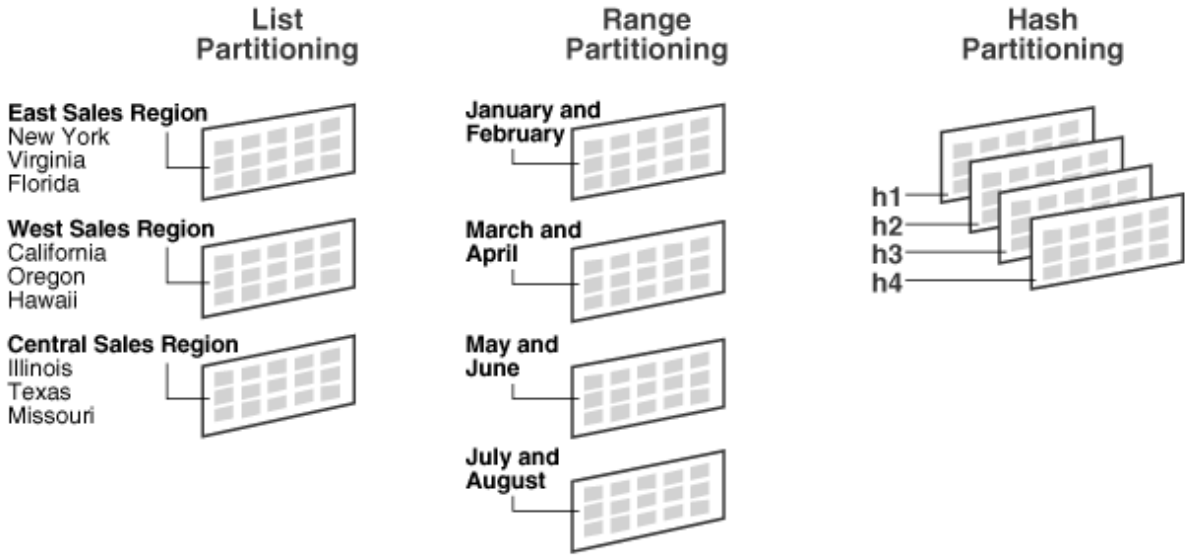
Description of "Figure 18-1 A View of Partitioned Tables"

Partition Key

Each row in a partitioned table is unambiguously assigned to a single partition. The partition key is a set of one or more columns that determines the partition for each row. Oracle automatically directs insert, update, and delete operations to the appropriate partition through the use of the partition key.

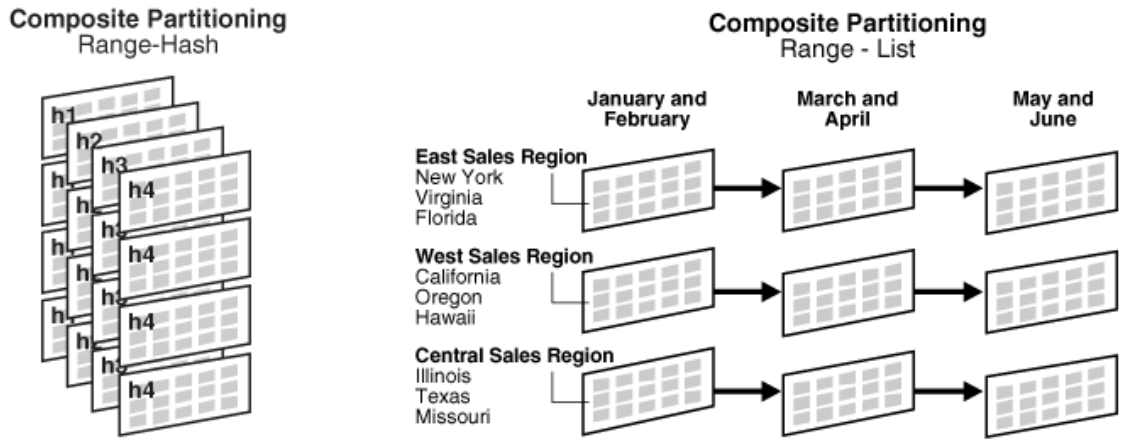
Oracle provides the following partitioning methods:

- Range Partitioning
- List Partitioning
- Hash Partitioning
- Composite Partitioning



Description of "Figure 18-2 List, Range, and Hash Partitioning"

Composite partitioning is a combination of other partitioning methods. Oracle supports range-hash and range-list composite partitioning. Following figures offers a graphical view of range-hash and range-list composite partitioning.



Description of "Figure 18-3 Composite Partitioning"

Range Partitioning

Partition by predefined ranges of continuous values.

Range partitioning maps data to partitions based on ranges of partition key values that you establish for each partition. It is the most common type of partitioning and is often used with dates. For example, you might want to partition sales data into monthly partitions.

For example:

The statement creates a table (sales_range) that is range partitioned on the sales_date field.

```
CREATE TABLE sales_range

(salesman_id  NUMBER(5),

salesman_name VARCHAR2(30),

sales_amount  NUMBER(10),

sales_date    DATE)

PARTITION BY RANGE(sales_date)

(

PARTITION sales_jan2000 VALUES LESS THAN(TO_DATE('02/01/2000','MM/DD/YYYY')),

PARTITION sales_feb2000 VALUES LESS THAN(TO_DATE('03/01/2000','MM/DD/YYYY')),

PARTITION sales_mar2000 VALUES LESS THAN(TO_DATE('04/01/2000','MM/DD/YYYY')),

PARTITION sales_apr2000 VALUES LESS THAN(TO_DATE('05/01/2000','MM/DD/YYYY'))

);
```

You can query individual table partition to get records only from the specific partition

```
select * from SALES_RANGE partition (sales_jan2000);
select * from SALES_RANGE partition (sales_feb2000
select * from SALES_RANGE partition (sales_mar2000);
select * from SALES_RANGE partition (sales_apr2000);
```

You can always query a partitioned table like a normal table too. In this case, the output will be from all the partitions inside the table

```
select * from SALES_RANGE;
```

List Partitioning

List partitioning enables you to explicitly control how rows map to partitions. You do this by specifying a list of discrete values for the partitioning key in the description for each partition. This is different from range partitioning, where a range of values is associated with a partition and from hash partitioning, where a hash function controls the row-to-partition mapping. The advantage of list partitioning is that you can group and organize unordered and unrelated sets of data in a natural way.

The details of list partitioning can best be described with an example. In this case, let's say you want to partition a sales table by region. That means grouping states together according to their geographical location as in the following example.

```
CREATE TABLE sales_list

(salesman_id  NUMBER(5),
```

```
salesman_name VARCHAR2(30),
sales_state   VARCHAR2(20),
sales_amount  NUMBER(10),
sales_date    DATE)
PARTITION BY LIST(sales_state)
(
PARTITION sales_west VALUES('California', 'Hawaii'),
PARTITION sales_east VALUES ('New York', 'Virginia', 'Florida'),
PARTITION sales_central VALUES('Texas', 'Illinois'),
PARTITION sales_other VALUES(DEFAULT)
);
```

Unlike range and hash partitioning, multicolumn partition keys are not supported for list partitioning.

Hash Partitioning

Hash partitioning enables easy partitioning of data that does not lend itself to range or list partitioning. Hash partitioning provides a method of evenly distributing data across a
specified number of partitions. Rows are mapped into partitions based on a hash value of the partitioning key

```
CREATE TABLE sales_hash
(salesman_id  NUMBER(5),
salesman_name VARCHAR2(30),
sales_amount  NUMBER(10),
week_no       NUMBER(2))
PARTITION BY HASH(salesman_id)
PARTITIONS 4
STORE IN (ts1, ts2, ts3, ts4);
```

The partitioning column is salesman_id, four partitions are created
and assigned system generated names, and they are placed in four named tablespaces (ts1, , ts2...).

Composite Partitioning

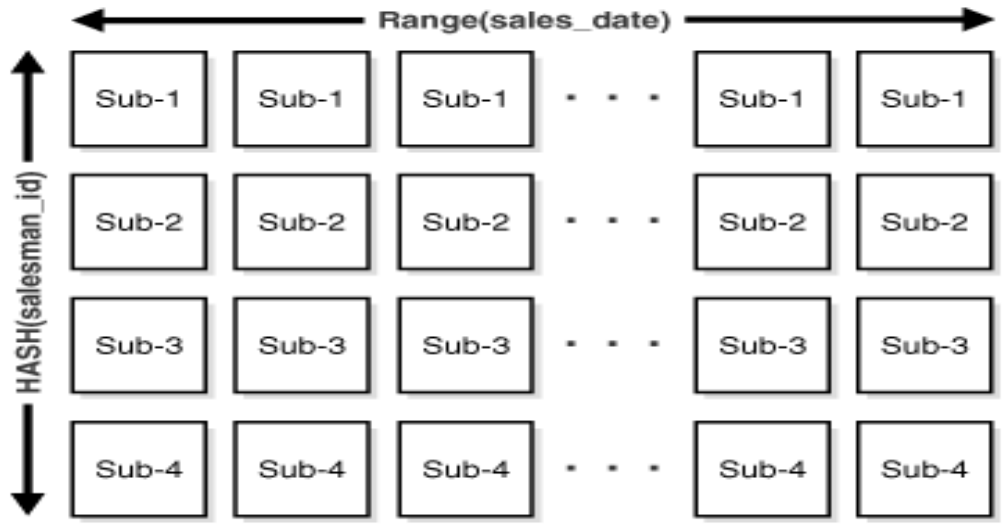
Composite partitioning partitions data using the range method, and within each partition, subpartitions it using the hash or list method.

Composite Partitioning Range-Hash Example

```
CREATE TABLE sales_composite

(salesman_id  NUMBER(5),

 salesman_name VARCHAR2(30),

 sales_amount  NUMBER(10),

 sales_date    DATE)

PARTITION BY RANGE(sales_date)

SUBPARTITION BY HASH(salesman_id)

SUBPARTITION TEMPLATE(

SUBPARTITION sp1 TABLESPACE ts1,

SUBPARTITION sp2 TABLESPACE ts2,

SUBPARTITION sp3 TABLESPACE ts3,

SUBPARTITION sp4 TABLESPACE ts4)

(PARTITION sales_jan2000 VALUES LESS THAN(TO_DATE('02/01/2000','MM/DD/YYYY'))

 PARTITION sales_feb2000 VALUES LESS THAN(TO_DATE('03/01/2000','MM/DD/YYYY'))

 PARTITION sales_mar2000 VALUES LESS THAN(TO_DATE('04/01/2000','MM/DD/YYYY'))

 PARTITION sales_apr2000 VALUES LESS THAN(TO_DATE('05/01/2000','MM/DD/YYYY'))

 PARTITION sales_may2000 VALUES LESS THAN(TO_DATE('06/01/2000','MM/DD/YYYY')));
```

This statement creates a table sales_composite that is range partitioned on the sales_date field and hash subpartitioned on salesman_id. When you use a template, Oracle names the subpartitions by concatenating the partition name, an underscore, and the subpartition name from the template. Oracle places this subpartition in the tablespace specified in the template. In the previous statement, sales_jan2000_sp1 is created and placed in tablespace ts1 while sales_jan2000_sp4 is created and placed in tablespace ts4. In the same manner, sales_apr2000_sp1 is created and placed in tablespace ts1 while sales_apr2000_sp4 is created and placed in tablespace ts4.

Following figure offers a graphical view of the previous example.



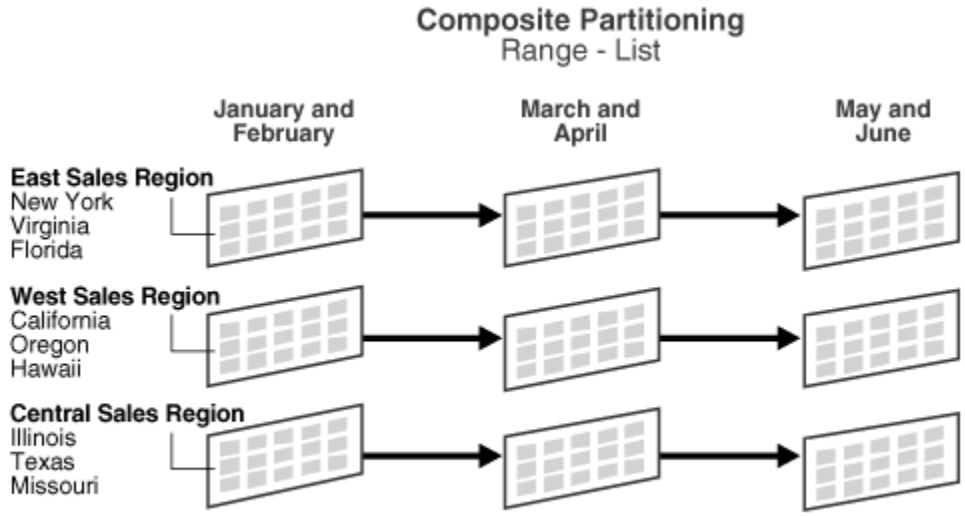
Description of "Figure 18-4 Composite Range-Hash Partitioning"

Composite Partitioning Range-List Example

```
CREATE TABLE bimonthly_regional_sales
(deptno NUMBER,
 item_no VARCHAR2(20),
 txn_date DATE,
 txn_amount NUMBER,
 state VARCHAR2(2))
PARTITION BY RANGE (txn_date)
SUBPARTITION BY LIST (state)
SUBPARTITION TEMPLATE(
SUBPARTITION east VALUES('NY', 'VA', 'FL') TABLESPACE ts1,
SUBPARTITION west VALUES('CA', 'OR', 'HI') TABLESPACE ts2,
SUBPARTITION central VALUES('IL', 'TX', 'MO') TABLESPACE ts3)
(
```

```
PARTITION janfeb_2000 VALUES LESS THAN (TO_DATE('1-MAR-2000','DD-MON-YYYY')),

PARTITION marapr_2000 VALUES LESS THAN (TO_DATE('1-MAY-2000','DD-MON-YYYY')),

PARTITION mayjun_2000 VALUES LESS THAN (TO_DATE('1-JUL-2000','DD-MON-YYYY'))

);
```

This statement creates a table bimonthly_regional_sales that is range partitioned on the txn_date field and list subpartitioned on state. When you use a template, Oracle names the subpartitions by concatenating the partition name, an underscore, and the subpartition name from the template. Oracle places this subpartition in the tablespace specified in the template. In the previous statement, janfeb_2000_east is created and placed in tablespace ts1 while janfeb_2000_central is created and placed in tablespace ts3. In the same manner, mayjun_2000_east is placed in tablespace ts1 while mayjun_2000_central is placed in tablespace ts3. Following figure offers a graphical view of the table bimonthly_regional_sales and its 9 individual subpartitions.



Description of "Figure 18-5 Composite Range-List Partitioning"

LISTING INFORMATION ABOUT PARTITION TABLES

You can query user_tab_partitions to get details about the table, partition name, number of rows in each partition and more

```
COLUMN high_value FORMAT A20
SELECT table_name,
       partition_name,
       high_value,
       num_rows
FROM   user_tab_partitions
ORDER BY table_name, partition_name;
```

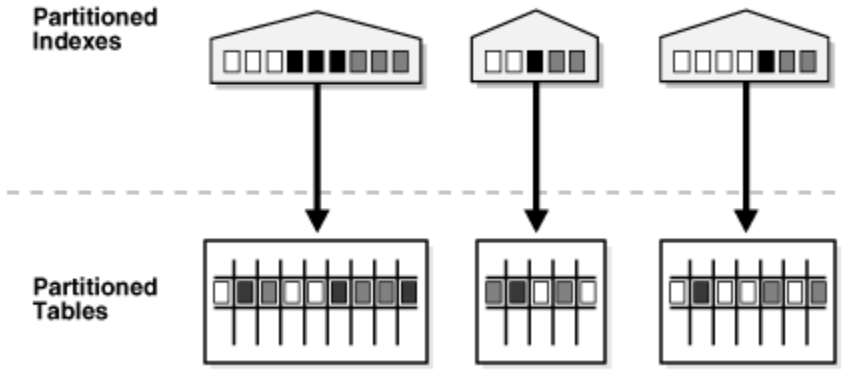
Create Partitioned Index

Just like partitioned tables, partitioned indexes improve manageability, availability, performance, and scalability. They can either be partitioned independently (global indexes) or automatically linked to a table's partitioning method (local indexes). In general, you should use global indexes for OLTP applications and local indexes for data warehousing or DSS applications.

You can create two types of indexes on a partition table:

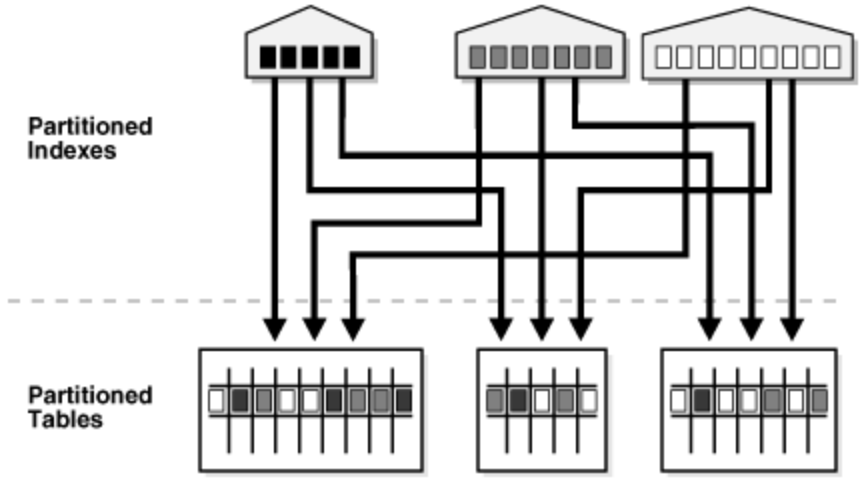
- Local Index and
- Global Index

Figure 18-6 Local Partitioned Index



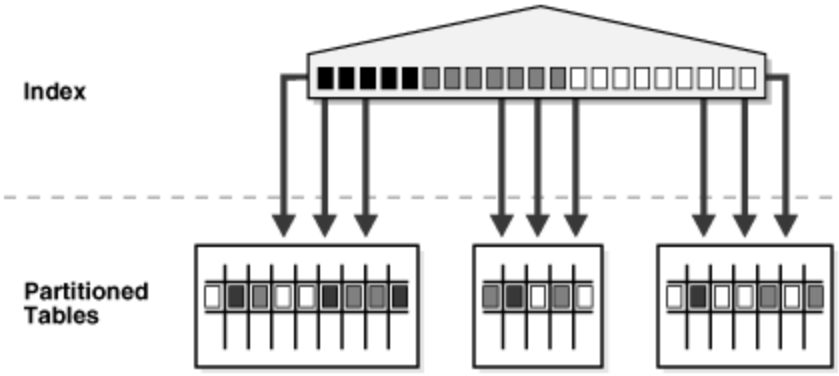
Description of "Figure 18-6 Local Partitioned Index"

Figure 18-7 Global Partitioned Index



Description of "Figure 18-7 Global Partitioned Index"

Figure 18-8 Global Nonpartitioned Index



Description of "Figure 18-8 Global Nonpartitioned Index"

You will be using **LOCAL** keyword in CREATE INDEX statement to create a local index

```
CREATE INDEX time_range_sales_indx ON TIME_RANGE_SALES(time_id) LOCAL
 (PARTITION year_1 TABLESPACE users,
  PARTITION year_2 TABLESPACE users,
  PARTITION year_3 TABLESPACE users,
  PARTITION year_4 TABLESPACE users);
```

To create a global index, you will be using **GLOBAL** keyword with CREATE INDEX

```
CREATE INDEX glob_time_range_sales ON TIME_RANGE_SALES(time_id)
GLOBAL PARTITION BY RANGE (time_id)
 (PARTITION glob_y1 VALUES LESS THAN (TO_DATE('01-JAN-1999','DD-MON-
YYYY')),
```

```
  PARTITION glob_y2 VALUES LESS THAN (TO_DATE('01-JAN-2000','DD-MON-
YYYY')),
  PARTITION glob_y3 VALUES LESS THAN (TO_DATE('01-JAN-2001','DD-MON-
YYYY')),
  PARTITION glob_y4 VALUES LESS THAN (MAXVALUE));
```

Check Partition Indexes

```
select INDEX_NAME, PARTITION_NAME from user_ind_partitions;
```